Experiment
'p_6_1_b_process_array_every_even_element_array' Results

June 5, 2026

Experiment outcome: SUCCESS
Bad responses: 0
Responses containing `assume` : 0
Responses containing `expect` : 0
Resolution attempts: 2
Hard fails (resolution): 0
Soft fails (resolution): 0
Verification attempts: 2

Problem Specification

Problem name: p_6_1_b_process_array_every_even_element_array
Natural language statement: null
Method signature: p_6_1_b_process_array_every_even_element_array(inputs: array<int>) returns (even_elements: array<int>)

Ensures

- `even_elements[..] == evens(inputs[..])`

Requires

- `inputs.Length == 10`

Functional Code Given

```
function evens(inputs: seq<int>): seq<int>
{
  if inputs ==[] then []
  else if inputs[0] % 2 == 0 then [inputs[0]] + evens(inputs[1..])
  else evens(inputs[1..])
}
```

GenAI interactions

Below you will find all interactions between the 'user' (program) and the 'assistant' (OpenAI).

Program → GenAI

```
You are given the following task to perform in Dafny:

Write a method to satisfy the below requirements.

The signature should be:

p_6_1_b_process_array_every_even_element_array(inputs: array<int>) returns (even_elements
    ↪ : array<int>)

The method should respect the following contract:

requires inputs.Length == 10, ensures even_elements[..] == evens(inputs[..])

The contract uses the following dafny code:

function evens(inputs: seq<int>): seq<int>
{
  if inputs ==[] then []
  else if inputs[0] % 2 == 0 then [inputs[0]] + evens(inputs[1..])
  else evens(inputs[1..])
}

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
    ↪ surround it. Do not show the signature nor contract. You must not use 'assume' nor
    ↪  'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnNXGBNpeFvgi5VzgWcFDNc9DjWZL
Model: gpt-5.5-2026-04-23
Created at: 1780660914
Finish reason: stop
Usage: 442 tokens in, and 6521 tokens out

```
{
  var s: seq<int> := [];

  if inputs[0] % 2 == 0 { s := s + [inputs[0]]; }
  if inputs[1] % 2 == 0 { s := s + [inputs[1]]; }
  if inputs[2] % 2 == 0 { s := s + [inputs[2]]; }
  if inputs[3] % 2 == 0 { s := s + [inputs[3]]; }
  if inputs[4] % 2 == 0 { s := s + [inputs[4]]; }
  if inputs[5] % 2 == 0 { s := s + [inputs[5]]; }
```

```
  if inputs[6] % 2 == 0 { s := s + [inputs[6]]; }
  if inputs[7] % 2 == 0 { s := s + [inputs[7]]; }
  if inputs[8] % 2 == 0 { s := s + [inputs[8]]; }
  if inputs[9] % 2 == 0 { s := s + [inputs[9]]; }

  even_elements := new int[|s|](i => s[i]);
  assert even_elements[..] == s;
}
```

Program → GenAI

```
When using dafny verify, the below error is emitted and verify fails:

problem_data/p_6_1_b_process_array_every_even_element_array/program.dfy(11,0): Error: a
    ↪ postcondition could not be proved on this return path
   |
11 | {
   | ^

problem_data/p_6_1_b_process_array_every_even_element_array/program.dfy(10,27): Related
    ↪ location: this is the postcondition that could not be proved
   |
10 |  ensures even_elements[..] == evens(inputs[..])
   | ^^

problem_data/p_6_1_b_process_array_every_even_element_array/program.dfy(25,38): Error:
    ↪ index out of range
   |
25 | even_elements := new int[|s|](i => s[i]);
   | ^


Dafny program verifier finished with 2 verified, 2 errors

Correct the error in code by altering only the method body. Produce and show only the
    ↪ Dafny body, including the curly braces that surround it. Do not show the signature
    ↪  nor contract. You must not use 'assume' nor 'expect' anywhere in your code.
```

GenAI → Program

System fingerprint: null
ID: chatcmpl-DnNaVq8rb1yRC15hNGBcJPbn8Y3Uo
Model: gpt-5.5-2026-04-23
Created at: 1780661115
Finish reason: stop
Usage: 1033 tokens in, and 8273 tokens out

```
{
  var s: seq<int> := [];
  var i: int := inputs.Length;
```

```
  assert inputs[i..] == [];
  assert evens(inputs[i..]) == [];

  while i > 0
    invariant 0 <= i <= inputs.Length
    invariant s == evens(inputs[i..])
  {
    i := i - 1;
    assert 0 <= i < inputs.Length;
    assert i + 1 <= inputs.Length;
    assert s == evens(inputs[i + 1..]);

    assert inputs[i..] != [];
    assert (inputs[i..])[0] == inputs[i];
    assert (inputs[i..])[1..] == inputs[i + 1..];

    if inputs[i] % 2 == 0 {
      assert evens(inputs[i..]) == [inputs[i]] + evens(inputs[i + 1..]);
      s := [inputs[i]] + s;
      assert s == [inputs[i]] + evens(inputs[i + 1..]);
      assert s == evens(inputs[i..]);
    } else {
      assert evens(inputs[i..]) == evens(inputs[i + 1..]);
      assert s == evens(inputs[i..]);
    }
  }

  assert i == 0;
  assert inputs[0..] == inputs[..];
  assert s == evens(inputs[..]);

  even_elements := new int[|s|](j => if 0 <= j < |s| then s[j] else 0);

  assert even_elements.Length == |s|;
  assert forall j :: 0 <= j < |s| ==> even_elements[j] == s[j];
  assert forall j :: 0 <= j < |s| ==> even_elements[..][j] == s[j];
  assert even_elements[..] == s;
}
```

Final Program

```
function evens(inputs: seq<int>): seq<int>
{
  if inputs ==[] then []
  else if inputs[0] % 2 == 0 then [inputs[0]] + evens(inputs[1..])
  else evens(inputs[1..])
}

method p_6_1_b_process_array_every_even_element_array(inputs: array<int>) returns (
    ↪ even_elements: array<int>)
```

```
      requires inputs.Length == 10
      ensures even_elements[..] == evens(inputs[..])
{
  var s: seq<int> := [];
  var i: int := inputs.Length;

  assert inputs[i..] == [];
  assert evens(inputs[i..]) == [];

  while i > 0
    invariant 0 <= i <= inputs.Length
    invariant s == evens(inputs[i..])
  {
    i := i - 1;
    assert 0 <= i < inputs.Length;
    assert i + 1 <= inputs.Length;
    assert s == evens(inputs[i + 1..]);

    assert inputs[i..] != [];
    assert (inputs[i..])[0] == inputs[i];
    assert (inputs[i..])[1..] == inputs[i + 1..];

    if inputs[i] % 2 == 0 {
      assert evens(inputs[i..]) == [inputs[i]] + evens(inputs[i + 1..]);
      s := [inputs[i]] + s;
      assert s == [inputs[i]] + evens(inputs[i + 1..]);
      assert s == evens(inputs[i..]);
    } else {
      assert evens(inputs[i..]) == evens(inputs[i + 1..]);
      assert s == evens(inputs[i..]);
    }
  }

  assert i == 0;
  assert inputs[0..] == inputs[..];
  assert s == evens(inputs[..]);

  even_elements := new int[|s|](j => if 0 <= j < |s| then s[j] else 0);

  assert even_elements.Length == |s|;
  assert forall j :: 0 <= j < |s| ==> even_elements[j] == s[j];
  assert forall j :: 0 <= j < |s| ==> even_elements[..][j] == s[j];
  assert even_elements[..] == s;
}
```

Total Token Usage

Input tokens: 1475
Output tokens: 14794
Reasoning tokens: 13876

Sum of 'total tokens': 16269

Experiment Timings

Overall Experiment started at 1780660914065, ended at 1780661323995, lasting 409930ms (409.93 seconds)
Iteration #1 started at 1780660914194, ended at 1780661115327, lasting 201133ms (201.13 seconds)
Iteration #2 started at 1780661115327, ended at 1780661323995, lasting 208668ms (208.67 seconds)